Multi-Threaded Web Server with IPC and Semaphores – User Manual

Authors: Diogo Martins 98501, Hélio Filho 93390
Course: Operating Systems (SO) 2025/26

Table of Contents

1. System Requirements
2. Installation
3. Compilation
4. Configuration
5. Execution
6. Testing
7. Troubleshooting

---

1. System Requirements

1.1 Operating System

- Linux (Ubuntu 20.04+ recommended)
- WSL2 on Windows (with performance considerations)

1.2 Required Packages

```
# Debian/Ubuntu
sudo apt-get update
sudo apt-get install -y build-essential gcc make

# Optional (for testing and debugging)
sudo apt-get install -y curl apache2-utils valgrind bc
```

1.3 Hardware Requirements

Resource	Minimum	Recommended
RAM	512 MB	2 GB
CPU	1 core	2+ cores
Disk	50 MB	100 MB

---

2. Installation

2.1 Download Project

```
# Clone or extract
project

cd
/path/to/your/workspace
# If using git:
git clone <repository-url> SO-
Project-2 cd SO-Project-2
```

2.2 Project Structure

```
SO-Project-2/
  src/                  # Source code
  www/                  # Web document
     root index.html
     errors/
         404.html
         500.html
  tests/                # Test scripts
  docs/                 # Documentation
  server.conf           # Configuration file
  Makefile              # Build system
```

---

3. Compilation

3.1 Standard Build

```
cd SO-Project-2
make
```

This produces the `server` executable in the project root.

3.2 Build Options

Command	Description
`make`	Standard build with warnings
`make debug`	Debug build (-g -O0)
`make release`	Optimized build (-O3)
`make clean`	Remove compiled files

3.3 Debug Build (for Valgrind)

```
make debug
```

3.4 Release Build (for Production)

```
make release
```

3.5 Verify Compilation

```
./server --help
```

Expected output:

```
Usage: ./server [OPTIONS]
  -c, --config PATH    Configuration file (default: ./server.conf)
  -p, --port PORT Override listen port
  -w, --workers NUM Number of worker processes
  -t, --threads NUM    Threads per worker
  -d, --daemon      Run in background (daemonize)
  -v, --verbose     Verbose startup logs
  -h, --help        Show this help

      --version     Show version
```

---

4. Configuration

4.1 Configuration File

Edit `server.conf` in the project root:

```
# Server Configuration
PORT=8080
DOCUMENT_ROOT=./www
NUM_WORKERS=4
THREADS_PER_WORKER=10
MAX_QUEUE_SIZE=100
LOG_FILE=access.log
CACHE_ENABLED=1
CACHE_SIZE_MB=10
TIMEOUT_SECONDS=30
```

4.2 Configuration Parameters

Parameter	Default	Description
PORT	8080	TCP port to listen on
DOCUMENT_ROOT	./www	Directory for static files
NUM_WORKERS	4	Number of worker processes
THREADS_PER_WORKER	10	Threads per worker
MAX_QUEUE_SIZE	100	Max pending connections per worker
LOG_FILE	access.log	Access log file path
CACHE_ENABLED	1	Enable file caching (0/1)
CACHE_SIZE_MB	10	Cache memory limit in MB
TIMEOUT_SECONDS	30	Connection timeout

4.3 Environment Variables (Override)

```
export
HTTP_PORT=9000
export HTTP_WORKERS=4
export
HTTP_THREADS=8
export
HTTP_TIMEOUT=60
./server
```

4.4 Configuration precedence

Configuration values are applied in the following order (later entries override earlier ones):

1. Built-in defaults (compiled into the binary)
2. server.conf file (if present)
3. Environment variables: `HTTP_PORT`, `HTTP_WORKERS`, `HTTP_THREADS`, `HTTP_TIMEOUT`
4. Command-line options (`-p`, `-w`, `-t`, `-c`) — highest priority

Note: `MAX_QUEUE_SIZE` is interpreted per worker. With the default of 4 workers and `MAX_QUEUE_SIZE=100` the server provides a total pending capacity of 400 connections; when a worker queue is full the server will return `503 Service Unavailable` for new connections.

4.4 Document Root Setup

Place your static files in the `www/` directory:

```
# Create test files
echo "<h1>Hello World</h1>" >
www/index.html echo "body { color:
blue; }" > www/styles.css
```

4.5 Custom Error Pages

Create custom error pages in `www/errors/`:

```
mkdir -p www/errors
echo "<h1>404 Not Found</h1>" >
www/errors/404.html echo "<h1>500 Server
Error</h1>" > www/errors/500.html
```

---

5. Execution

5.1 Start Server (Foreground)

```
./server
```

Output (minimal):

```
[WORKER 0] started
[WORKER 1] started
[WORKER 2] started
[WORKER 3] started
[MASTER] Listening on port 8080...
```

Output (verbose mode `-v`):

```
[MAIN] Starting server
initialization... [MAIN] Shared
memory created.
[MAIN] Config: port=8080 workers=4 threads=10 cache=10485760 bytes timeout=30
docroot=./www [MAIN] Semaphores initialized.
[MAIN] Forking 4
workers... [WORKER 0]
started
[WORKER 1] started
[WORKER 2] started
[WORKER 3] started
```

5.2 Start Server (Background)

```
./server -d
# or
./server &
```

5.3 Start with Custom Options

```
# Custom port
./server -p 9000

# Custom workers and threads
./server -w 4 -t 8
```

```
# Custom config file
./server -c /path/to/custom.conf

# Combined options
./server -p 9000 -w 4 -t 8 -d
```

5.4 Verify Server is Running

```
# Check if listening
ss -ltnp | grep :8080

# Test with curl
curl http://localhost:8080/

# Check HTTP headers
curl -I http://localhost:8080/
```

5.5 Stop Server

```
# Graceful shutdown (SIGTERM)
pkill -f ./server

# Or find PID and kill
pgrep -f
./server kill
<PID>

# Force kill if needed
pkill -9 -f ./server
```

5.6 View Access Logs

```
tail -f access.log
```

Log format (Apache Combined):

```
192.168.1.1 - - [11/Dec/2025:10:00:00 +0000] "GET /index.html HTTP/1.1" 200 1234 "-"
"curl/7.68.0"
```

5.7 CGI Support (bonus)

The server supports CGI scripts placed inside the document root (for example `www/test.cgi`). CGI scripts are executed via `fork()/exec()` and receive standard CGI environment variables such as `REQUEST_METHOD`, `QUERY_STRING`, and `SCRIPT_NAME`.

Example:

```
curl "http://localhost:8080/test.cgi?name=value"
```

The server will return the script output as the HTTP response body.

5.8 Real-Time Dashboard (bonus)

A real-time dashboard is served at `/dashboard.html` and the underlying JSON API is available at `/api/stats`.

Open in browser:

```
http://localhost:8080/dashboard.html
```

Programmatic access to stats:

```
curl http://localhost:8080/api/stats
# Returns JSON with fields such as total_requests, bytes_transferred,
cache_hits, cache_misses, uptime_
```

5.9 Keep-Alive (persistent connections)

The server supports HTTP/1.1 persistent connections (Keep-Alive). Clients may reuse the same TCP connection for multiple requests. Idle connections are closed after `TIMEOUT_SECONDS` (default 30). To adjust the timeout, change `TIMEOUT_SECONDS` in `server.conf`.

---

6. Testing

6.1 Quick Test

```
# Start server in one terminal
./server

# In another terminal
curl http://localhost:8080/
curl http://localhost:8080/index.html
curl  http://localhost:8080/nonexistent.html # Should return 404
```

6.2 Run Test Suite

```
cd tests

# Fix line endings (WSL/Windows)
sed -i 's/\r$//'
*.sh chmod +x *.sh

# Run all tests
./run_all_tests.sh

# Run individual test suites
./functional_tests.sh
./concurrency_tests.sh
./synchronization_tests.sh
./stress_tests.sh
```

6.3 Load Testing with Apache Bench

```
# 1000 requests, 10 concurrent
ab -n 1000 -c 10 http://localhost:8080/index.html

# 10000 requests, 100 concurrent
ab -n 10000 -c 100 http://localhost:8080/index.html
```

6.4 Memory Leak Testing

```
# Build with debug symbols
make debug
```

```
# Run with Valgrind
valgrind --leak-check=full --log-file=valgrind.log ./server

# In another terminal, generate some load
ab -n 1000 -c 10 http://localhost:8080/

# Stop server and check valgrind.log
```

6.5 Race Condition Testing

```
# Run with Helgrind
valgrind --tool=helgrind --log-file=helgrind.log ./server

# Generate concurrent load
ab -n 1000 -c 50 http://localhost:8080/

# Check helgrind.log for race conditions
```

---

7. Troubleshooting

7.1 "Address already in use"

Problem: Server fails to start with `bind(): Address already in use`

Solution:

```
# Find and kill process using the port
sudo fuser -k 8080/tcp
# or
sudo lsof -i
:8080 kill
<PID>

# Wait for TIME_WAIT to expire or use different port
./server -p 8081
```

7.2 "Permission denied"

Problem: Cannot bind to port < 1024

Solution:

```
# Use port >= 1024
./server -p 8080

# Or run as root (not recommended)
sudo ./server -p 80
```

7.3 "Cannot execute: required file not found"

Problem: Script has Windows line endings (CRLF)

Solution:

```
sed -i 's/\r$//'
script.sh chmod +x
script.sh
```

7.4 Slow Performance on WSL

Problem: Very slow file operations on `/mnt/c/...` . (this entire project was done on Windows, using WSL2 Server on VS Code)

Solution: Copy project to native Linux filesystem:

```
cp -r /mnt/c/path/to/SO-Project-2 ~/SO-
Project-2 cd ~/SO-Project-2
make clean && make
./server
```

7.5 Server Not Responding

Diagnosis:

```
# Check if process is running
pgrep -a server

# Check if port is listening
ss -ltnp | grep 8080

# Check server logs
tail -50 access.log

# Test locally
curl -v http://localhost:8080/
```

7.6 High Memory Usage

Diagnosis:

```
# Monitor memory
top -p $(pgrep -d, -f ./server)

# Check for leaks
valgrind --leak-check=full
```

`./server` **Solution**: Reduce cache size in

`server.conf`: `CACHE_SIZE_MB=5`

7.7 Connection Timeouts

Solution: Increase timeout in `server.conf`:

`TIMEOUT_SECONDS=60`

Quick Reference

Start Server

```
make && ./server
```

Stop Server

```
pkill -f ./server
```

Test Server

```
curl http://localhost:8080/
```

Run Tests

```
cd tests && ./run_all_tests.sh
```

Check Logs

```
tail -f access.log
```

Glossary of Technical Terms

- **API**: Application Programming Interface; here refers to the HTTP JSON endpoint `/api/stats`.
- **CGI**: Common Gateway Interface — a mechanism to execute scripts (e.g., `www/test.cgi`) and return their output as HTTP responses.
- **Daemonize**: Run the server in the background (detached from terminal), typically via the `-d` option.
- **HTTP/1.1**: The version of the HTTP protocol supported. Includes persistent connections (Keep-Alive).
- **Keep-Alive**: HTTP feature that allows multiple requests over a single TCP connection; idle connections are closed after `TIMEOUT_SECONDS`.
- **JSON**: JavaScript Object Notation; format used by `/api/stats` for statistics.
- **LRU**: Least Recently Used — cache eviction policy used by the file cache.
- **MAX_QUEUE_SIZE**: Maximum pending connection slots per worker; default `100` (per worker).
- **MIME**: Multipurpose Internet Mail Extensions — used to indicate content types (e.g. `text/html`, `image/png`).
- **POSIX semaphore (sem_t)**: Inter-process synchronization primitive used here to protect shared statistics and queues.
- **pthread_rwlock_t (rwlock)**: POSIX reader-writer lock used in the cache to allow multiple readers or one writer.
- **SCM_RIGHTS**: Unix-domain socket ancillary message type used to pass file descriptors between processes.
- **sendfile()**: System call used to efficiently transfer file contents from disk to network socket.
- **Shared memory**: Memory region mapped into multiple processes (master + workers) used for queues and stats.
- **SIGINT / SIGTERM**: Signals used to request graceful shutdown (Ctrl+C sends SIGINT).
- **Valgrind**: Tool for memory debugging; used to verify no leaks.
- **Helgrind**: Valgrind tool for detecting data races in threaded programs.
- **Makefile**: Build script used by `make` to compile and manage targets.
- **NUM_WORKERS / THREADS_PER_WORKER**: Number of worker processes and threads per worker (defaults: 4 workers, 10 threads each).
- **Document root**: The `www/` directory where static files, errors, and CGI scripts live.
- **Daemon**: Background process; the server can be run in the background using `-d`.
- **503 Service Unavailable**: HTTP response when the server cannot accept new connections because worker queues are full.